

Two-Step Contextual Enrichment with Hyper-Dimensional Anchors: Discrete Control Prefixes for Inference-Time Reliability

Kaleb Cadenhead
Automation Optimization
kcadenhead@automationoptimization.com

April 27, 2026

Abstract

Many language-model failures begin before the first visible token: the model starts decoding from the wrong conditional state. Two-Step Contextual Enrichment (TSCE) is a discrete inference-time control method for improving reliability without changing model weights. It first generates a Hyper-Dimensional Anchor (HDA), then decodes the final answer under that anchor. The HDA is the steering prefix under test: an ordinary input-token sequence selected to shift the model’s pre-decode state into a better task basin before visible answer generation.

Operationally, TSCE is a wrapper around existing chat-completion calls. It adds an anchor-generation step and a small final-prompt control prefix, without model retraining or activation-patching infrastructure. This makes the deployment cost explicit: one extra anchor call when anchors are generated online, plus ordinary prompt-token overhead during the final decode.

Across 17 comparable black-box evaluations, the paired baseline-vs-best summary improves from 2093/3330 solved instances (62.9%) to 2736/3330 solved instances (82.2%), with 10,460 condition-level scored outputs across 64 pass-rate rows. This is an upper-envelope run-level summary across heterogeneous HDA policies, not a single frozen recipe. A separate 100-task fixed-policy Gemma validation with compact `latent_bias` anchors is neutral: baseline and TSCE both solve 66/100 tasks. On 69 Gemma prompts selected because baseline decoding failed, a 14,421-row format-robustness sweep shows that several noncanonical HDA surface forms still rescue many failures, while removing the final-answer scaffold largely collapses the effect.

In a controlled token-causality evaluation, task-relevant HDA strings lift the target-vs-distractor pre-generation probe margin by +1.1629, while tokenizer-matched random controls remain near zero (−0.0593, −0.0011, −0.0231). A naturalistic story-theme probe shows the same measurement pattern for *forgiveness after betrayal*: controlled theme-proxy HDA strings lift the pre-generation margin by +2.5849, while matched random controls do not. A larger result corpus supports anchor generation, policy development, and mechanistic diagnostics; its inventory is separated from accuracy claims in Appendix A.

1 Introduction

TSCE is a discrete inference-time control method. It treats ordinary input tokens as a control interface, not merely as text for instruction following. The HDA is the causal steering prefix under test: a compact token sequence generated before the final answer and used to move the model’s pre-decode state into a more useful task basin.

Language-model failures often begin before the first visible token is emitted. A model may have the relevant capability but enter the final decode in a broad, weakly constrained region of its conditional distribution. In that state, repeated samples drift, formatting constraints are missed, and small prompt ambiguities become large output differences. This framing is consistent with the prompt sensitivity observed in few-shot language models and with the practical difficulty of evaluating free-form model outputs reliably [2, 7].

TSCE addresses that failure mode by inserting a learned or sampled token intervention before the final answer. The intervention is the HDA. Its role is not to explain the task to a human. Its role is to bias the model’s internal trajectory so that the final decode starts from a narrower and more task-appropriate region.

This paper evaluates the following claim:

TSCE improves reliability when the HDA shifts the model’s pre-decode state into a better task basin.

The evaluation is organized around three forms of evidence. First, outcome benchmarks measure downstream reliability under TSCE conditioning. Second, controlled token-causality tests measure target-relevant probability movement before long-form generation. Third, proxy-family and activation-space diagnostics characterize which token neighborhoods produce the measured shift. The limitations section bounds the claim to cases in which the HDA produces task-relevant pre-decode movement.

2 Method

Let X be the user prompt, A be the HDA token sequence, and Y be the final answer. TSCE has two phases:

$$A \sim P_{\phi}(A \mid X, S_{\text{anchor}}), \tag{1}$$

$$Y \sim P_{\theta}(Y \mid X, A, S_{\text{final}}). \tag{2}$$

The method is model-agnostic because A is passed through the same token interface as any other prompt prefix. No weights are changed and no hidden activations are patched by hand. The intervention still has a latent effect because every token is embedded, attended to, and transformed into high-dimensional hidden state before decoding. This places TSCE near work on code-based neurosymbolic reasoning, adversarial gibberish-prompt effects, latent-action control, and continuous-latent reasoning, but only as neighboring design context. Those sources motivate the plausibility of token-mediated or representation-mediated control; the evidence for TSCE itself is the outcome and pre-decode measurement reported below [1, 3, 6, 4, 16].

2.1 Operational Deployment Profile

The practical deployment shape is intentionally simple. TSCE can be inserted as a two-call wrapper around an existing chat-completion pipeline: phase 1 generates or retrieves an HDA, and phase 2 sends the original request with the HDA placed in the final conditioning context. The runtime cost is explicit: one additional anchor-generation call when anchors are generated online, plus the token overhead of the HDA and fixed scaffold in the final prompt. If anchors are precomputed, cached, or supplied by an external policy, the integration reduces to a single final call with a control prefix.

The reference Python implementation includes a `TSCEChat` wrapper in `tsce_agent_demo/tsce_chat.py`. The wrapper accepts either a plain string prompt or an OpenAI-style chat message

array, supports OpenAI, Azure OpenAI, Ollama, and a local phase-2 backend path, and records phase-level request, response, usage, and latency metadata for operational logging. It can also accept `force_anchor`, which allows a separate anchor service or learned policy to be dropped in without changing the final-answer application code. The implementation lineage has been used by Automation Optimization since 2023 in applied automation work; that operational history motivates the wrapper design, while the benchmark and causality claims in this paper remain limited to the measured artifacts reported below.

The intended adoption pattern is an A/B test around existing prompts, not a replacement for the surrounding reliability stack. TSCE is most relevant when failures look like underconstrained decoding, format drift, or brittle instruction adherence. In those cases, teams can compare baseline, TSCE, and matched controls while directly measuring added tokens, latency, and task-level acceptance rates.

2.2 HDA Generation Policy

The evidence in this paper uses two distinct HDA sources. First, the black-box reliability subset contains heterogeneous runs: some use sampled prompt-family anchors, some use generated or searched anchors, and some evaluate multiple TSCE/HDA variants. For that reason the paired baseline-vs-best result is an upper-envelope run summary. It asks whether each run contained a TSCE/HDA condition that produced the predicted reliability shift; it should not be read as the performance of a single fixed production recipe.

Second, newer Gemma evaluations use a reproducible prompt-family policy. The phase-1 model samples multiple candidate anchors from one or more prompt families: `opaque_control`, `latent_bias`, `task_abstract`, and `keyboard_drift`. The default family is `latent_bias`. Candidates are canonicalized as `<HDA>...</HDA>` strings and selected by a heuristic score that favors compact alphabetic tokens, high uniqueness, low repetition, low overlap with the user prompt, absence of digits or punctuation, absence of instruction vocabulary, and low sentence-likeness. This draws on the same premise as control-code conditioning and pause-token delayed computation: pre-output tokens can change the distribution and computation used for the following decode [10, 15]. The Gemma comparison path records the prompt family, selector, candidate count, selected candidate, candidate features, source mode, anchor temperature, and top- p in each run. Default sampling uses anchor temperature 1.25, top- $p = 0.95$, and the heuristic selector; final answer decoding is run at low temperature where deterministic comparison is required.

This policy description is separate from learned HDA policies. Diffusion and reinforcement-learning policies are engineering routes for discovering better anchors, but the mechanism claim only requires an ordinary-token prefix that produces measurable pre-decode movement. Appendix C gives the policy summary used for reproduction.

2.3 Terminology and Related Methods

The term Hyper-Dimensional Anchor is literal.

- It is an *anchor* because it pulls the second pass toward a region of behavior before the answer begins.
- It is *hyper-dimensional* because the steering effect is mediated through the model’s embedding, attention, and residual-stream geometry rather than through a single human-readable instruction.
- It is a *token sequence* because TSCE uses ordinary input tokens as the control channel.

TSCE is related to prior work on control prefixes and latent control, but the intended distinction is functional: TSCE uses ordinary tokens as a control prefix optimized for internal steering, not as a human-readable explanation or final-answer draft. The closest prior work provides analogies and boundary cases rather than direct validation: code-based neurosymbolic reasoning shows that non-natural-language structure can support reasoning, adversarial gibberish-prompt work shows that semantically odd strings can affect model behavior, and latent-action or continuous-latent methods show related ways of biasing computation through compact internal representations [1, 3, 6, 4, 16].



Mechanistic test: perturb HDA tokens and measure probe/logit movement before long-form decoding.

Figure 1: The HDA is the proposed mediator in TSCE: it is generated before the visible answer and shifts the model’s pre-decode state through the normal token pathway.

3 Hypotheses

The mechanism implies five testable hypotheses.

1. **Outcome lift:** conditioning on HDA strings should improve task reliability when the HDA moves the model toward useful constraints.
2. **Pre-decode movement:** the effect should be measurable before long-form generation, not only after grading final answers.
3. **Matched controls:** tokenizer-matched random strings should not reproduce the same target-relevant movement.
4. **Surface-form robustness:** some effect should survive non-semantic perturbations of an effective HDA—including shuffling, deletion, wrapper changes, line breaks, and tag changes—when the modified sequence remains in a similar conditioning basin.
5. **Compositionality:** semantically relevant proxy neighborhoods should move the model more than neutral or irrelevant token neighborhoods.

The experiments below are organized around these hypotheses.

4 Outcome Evaluation

Black-box task evaluations estimate the downstream effect of HDA conditioning on final-answer accuracy. Each evaluation compares a baseline condition against one or more TSCE/HDA conditions. Table 1 shows a 300-task GPT-3.5 evaluation that includes single-pass, Chain-of-Thought,

Self-Refine, and TSCE-conditioned variants. These comparison points represent verbal and agentic test-time scaffolds rather than hidden control prefixes [13, 12, 8, 14].

Method	Passes	Pass rate
Baseline	147/300	49.0%
Chain-of-Thought	186/300	62.0%
Self-Refine	163/300	54.3%
TSCE	238/300	79.3%
CoT+TSCE	240/300	80.0%
Refine+TSCE	242/300	80.7%

Table 1: Outcome-level reliability on the 300-task GPT-3.5 evaluation. The TSCE-conditioned conditions outperform the corresponding non-TSCE baselines, indicating that HDA conditioning contributes reliability beyond the additional inference pass alone.

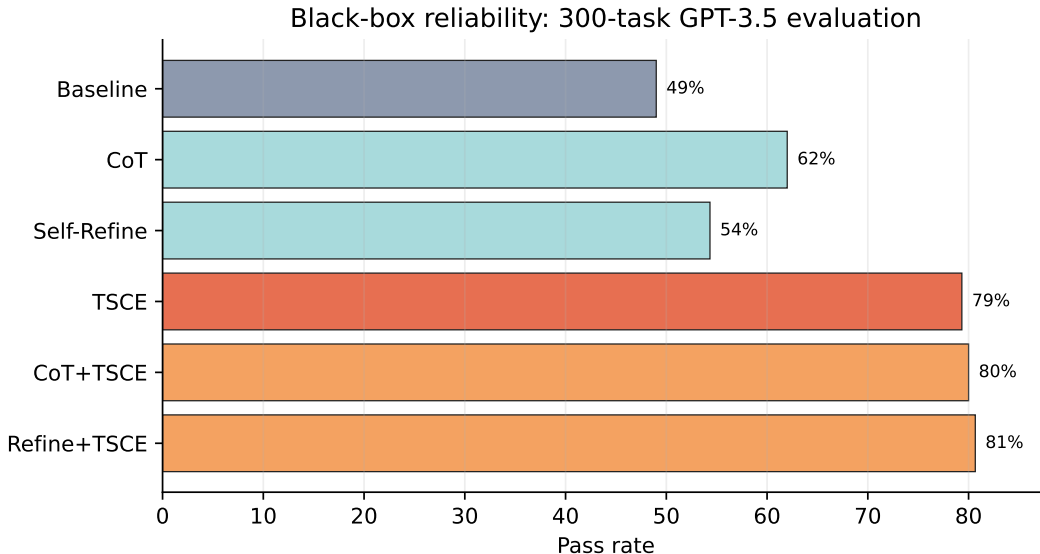


Figure 2: Outcome-level pass rates for the 300-task GPT-3.5 evaluation.

The comparable black-box reliability subset contains 17 run-level evaluations that include pass counts for a baseline and at least one TSCE/HDA condition. Across all pass-rate rows, this subset contains 64 condition rows and 10,460 condition-level scored outputs. The aggregate baseline-vs-best comparison is a paired run-level upper-envelope summary: it selects one baseline row and one best TSCE/HDA row per run, producing a 3,330-instance denominator on each side of the comparison. Under that paired summary, baseline conditions solve 2093/3330 instances (62.9%), while the best TSCE/HDA condition within each run solves 2736/3330 instances (82.2%). Because run configurations, model settings, task slices, and HDA generation policies are not identical, this is framed as a cross-run reliability summary rather than as an independence-weighted meta-analysis or a fixed-policy replication result. The behavioral pattern is consistent with the proposed mechanism: 15 of 17 run-level evaluations show positive lift, 12 of 17 show at least +15 percentage points, and the median best-condition lift is +22.0 percentage points.

Cross-run measure	Value
Run-level black-box evaluations	17
Pass-rate condition rows	64
Condition-level scored outputs	10,460
Paired baseline-vs-best task instances	3,330 per side
Aggregate baseline accuracy over paired instances	2093/3330 (62.9%)
Aggregate best TSCE/HDA accuracy over paired instances	2736/3330 (82.2%)
Evaluations with positive lift	15/17
Evaluations with lift ≥ 15 percentage points	12/17
Median best-condition lift	+22.0 percentage points

Table 2: Cross-run summary of black-box evaluations. Best-condition aggregation asks whether any TSCE/HDA configuration in each run produced the predicted downstream reliability shift. It is an upper-envelope summary across heterogeneous setups; fixed-policy replication should report each chosen HDA generation policy separately rather than selecting the best condition after evaluation.

To check the cherry-picking risk directly, we also ran a fixed-policy Gemma validation. The policy was selected before evaluation: Gemma E4B, task kind `auto`, 100 generated tasks, seed 260426, `latent_bias` anchor family, heuristic source mode, heuristic selector, two anchor candidates, compact 4–8 token anchors, anchor temperature 1.25, top- $p = 0.95$, and final-answer temperature 0.01. This fixed policy does not reproduce the upper-envelope lift. TSCE ties the baseline at 66/100, with four TSCE-only wins and four baseline-only wins. The matched opaque controls are also close: random-valid anchors solve 65/100, shuffled anchors solve 64/100, and context-collision anchors solve 66/100. This run disables activation capture, so it is an outcome-only fixed-policy check. The result is useful precisely because it is negative: the upper-envelope summary shows that some HDA policies can help, but this compact fixed policy does not by itself establish fixed-recipe outcome improvement.

Condition	Passes	Pass rate
Baseline	66/100	66.0%
Readable control	64/100	64.0%
TSCE compact <code>latent_bias</code>	66/100	66.0%
Random-valid matched anchor	65/100	65.0%
Shuffled anchor	64/100	64.0%
Context-collision anchor	66/100	66.0%

Table 3: Fixed-policy Gemma validation on a 100-task auto slice. Unlike the baseline-vs-best summary, this table freezes one compact HDA policy before evaluation and reports all matched controls.

4.1 Format Robustness on Baseline Failures

The fixed-policy result above asks whether one compact HDA recipe improves a mixed 100-task slice. A separate format-robustness sweep asks a narrower question: when baseline decoding already fails, is the effect tied to one exact `<HDA>... </HDA>` surface form? We evaluated Gemma E4B on 69 prompts selected from baseline failures, using 16 candidate anchors per prompt, 13 HDA format variants, and an empty-system baseline. Final-answer decoding used temperature 0.01 and

$\text{top-}p = 0.95$, producing 14,421 scored rows.

Condition	Passes	Pass rate	Oracle@16
Baseline empty system	0/69	0.0%	—
One token per line	614/1104	55.6%	65.2%
User prefix	599/1104	54.3%	63.8%
Control tag	540/1104	48.9%	52.2%
Canonical inline	534/1104	48.4%	55.1%
User suffix	513/1104	46.5%	58.0%
Multi-block HDA	512/1104	46.4%	53.6%
System no scaffold	53/1104	4.8%	24.6%

Table 4: Gemma E4B format-robustness sweep on 69 baseline-failure prompts. Multiple noncanonical HDA surface forms preserve substantial rescue, while placing the anchor in the system message without the final-answer scaffold largely collapses the effect.

This separates payload robustness from scaffold invariance. The result argues against the narrow explanation that TSCE only found a brittle XML-like prompt trick: line-wrapped, control-tagged, user-prefixed, and multi-block variants all preserve substantial rescue. At the same time, the weak `system_no_scaffold` condition shows that the anchor is not an arbitrary string that works wherever it is placed. The HDA must be consumed in a conditioning context that influences the final decode.

We distinguish comparable accuracy trials from supporting generation, policy, and mechanistic artifacts. The broader result tree contains 479 files and 100,816 counted rows/records/log lines; detailed accounting is provided in Appendix A.

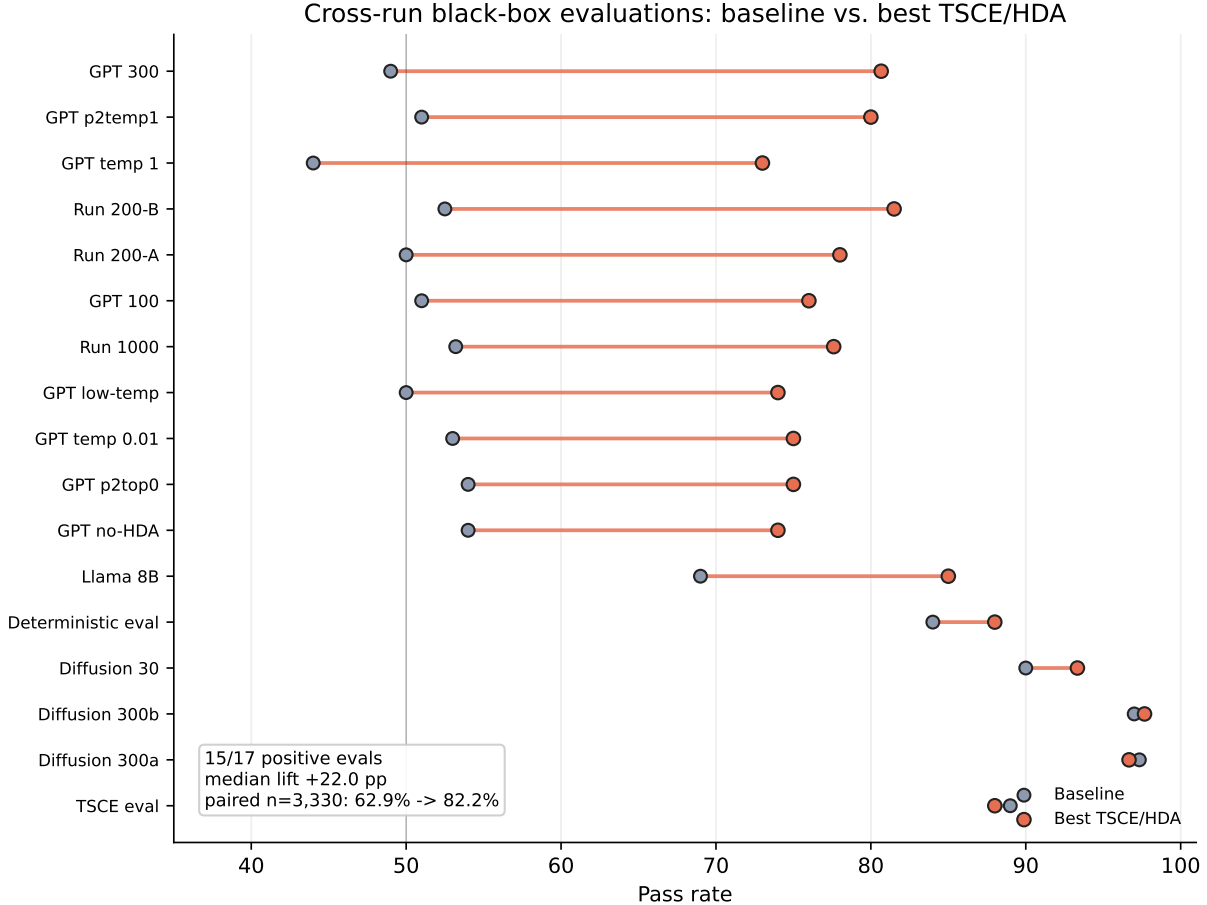


Figure 3: Cross-run black-box evaluation summary. Most run-level evaluations show positive lift from baseline to the best TSCE/HDA condition; high-ceiling evaluations show smaller or neutral effects when baseline performance is already near saturation or when the HDA does not produce a beneficial pre-decode shift.

The variation across evaluations is relevant to the mechanism rather than incidental. TSCE is expected to improve reliability when the generated HDA moves the model toward task-relevant constraints, and to produce smaller gains when the model, task, or anchor-generation procedure does not create a useful pre-decode shift. That distinction also separates the present evidence from majority-vote or search-heavy test-time compute methods, which can improve outcomes through additional sampling, acting, or refinement rather than through a measured pre-decode control state [12, 8, 14].

5 Controlled Token-Causality Evaluation

The controlled token-causality evaluation tests the proposed mediator directly. A held-out composite target phrase is used as the target concept:

a greenhouse full of sleeping glass birds

The HDA is constrained not to name the target phrase. Instead, controlled HDA strings are constructed from indirect proxy families: glass/light, avian/sky, garden/green, and sleep/quiet.

The measured variable is a pre-generation target-vs-distractor probe margin, defined as target probe log-probability minus negative subject probe log-probability. Because this quantity is measured before long-form generation, it tests the pre-decode state rather than post hoc answer quality.

Condition	Mean margin	Lift vs. no HDA
No HDA	-3.7140	0.0000
HDA	-2.5511	+1.1629
Shuffled HDA	-2.7742	+0.9398
Head-deleted HDA	-2.8808	+0.8332
Mid-deleted HDA	-2.6698	+1.0441
Tail-deleted HDA	-2.6374	+1.0766
Neutral-replaced HDA	-2.8363	+0.8777
Random matched 1	-3.7733	-0.0593
Random matched 2	-3.7151	-0.0011
Random matched 3	-3.7371	-0.0231

Table 5: Controlled token-causality conditions. Task-relevant HDA strings move the pre-decode target margin, while tokenizer-matched random strings remain near the no-HDA baseline.

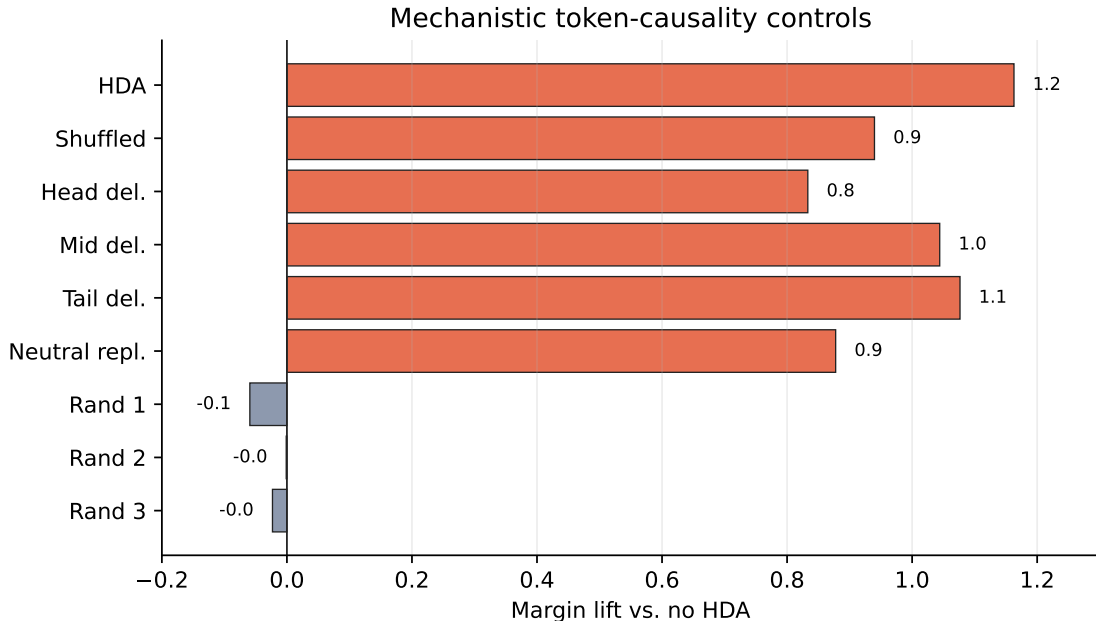


Figure 4: Pre-generation probe-margin lift under controlled HDA perturbations.

The result supports the mediator role assigned to the HDA. The controlled HDA condition increases the pre-generation margin by +1.1629 relative to no-HDA, while three tokenizer-matched random controls produce lifts near zero. This separates task-relevant token steering from the mere presence of additional prefix text. Under the proposed mechanism, reliability improves when this pre-decode movement shifts probability mass toward outputs that better satisfy the task. In information-theoretic terms, conditioning on an informative variable reduces uncertainty in expectation; this is the narrow sense in which the anchor acts as an entropy-reducing intervention before

the final sample is drawn [11].

5.1 Naturalistic Theme Probe

The greenhouse target is deliberately artificial: it gives strong experimental control, but it is not representative of everyday task semantics. To test whether the same probe construction extends to a more natural concept, a second token-causality evaluation uses the story theme *forgiveness after betrayal*. The positive probes include the exact theme, the facets forgive, betray, and trust, and a narrative-frame probe. Negative probes are other story themes: homecoming after exile, memory as inheritance, and freedom purchased by sacrifice.

Condition	Mean margin	Lift vs. no HDA
No HDA	-0.3342	0.0000
HDA	2.2507	+2.5849
Shuffled HDA	2.1973	+2.5314
Context-collision HDA	-0.3485	-0.0144
Random matched 1	-0.8837	-0.5496
Random matched 2	-0.4713	-0.1372
Random matched 3	-0.1162	+0.2180

Table 6: Naturalistic theme token-causality evaluation for *forgiveness after betrayal*. Controlled theme-proxy HDA strings move the pre-decode margin toward the target theme, while context-collision and tokenizer-matched random controls do not reproduce the effect.

This result is not a substitute for broad natural-task validation, but it addresses the main concern raised by the artificial greenhouse target: the same pre-generation probe trick can detect target-relevant movement for a natural story theme. The effect is also directionally specific. The controlled HDA lift is +2.5849, shuffled anchors preserve most of the movement, the context-collision control is essentially unchanged from no-HDA, and matched random controls do not produce a comparable lift.

6 Proxy-Family Structure

The controlled perturbations clarify which properties of the HDA are relevant to the observed effect.

First, the effect is not caused by arbitrary prefix text. Tokenizer-matched random strings produce near-zero lift, indicating that the token sequence must land in a useful region of the model’s conditional space.

Second, the effect is not tied to one exact surface form. Shuffled and deletion variants preserve much of the lift, implying an equivalence class of useful token sequences whose different surface forms can activate a similar semantic basin.

The baseline-failure format sweep in Table 4 supports the same interpretation at the downstream task level: several surface realizations of the same candidate anchor payload remain effective, while removing the final-answer scaffold can destroy most of the effect.

Third, the effect is compositional. With total proxy-token budget fixed, proxy-family coverage helps, but composition matters. Coverage across all four target families gives mean lift +1.9312, while the glass/light + avian/sky pair gives +2.2404. Family-presence deltas identify the

strongest neighborhoods for this target as glass/light +0.7247 and avian/sky +0.7078, followed by garden/green +0.3247 and sleep/quiet +0.1412.

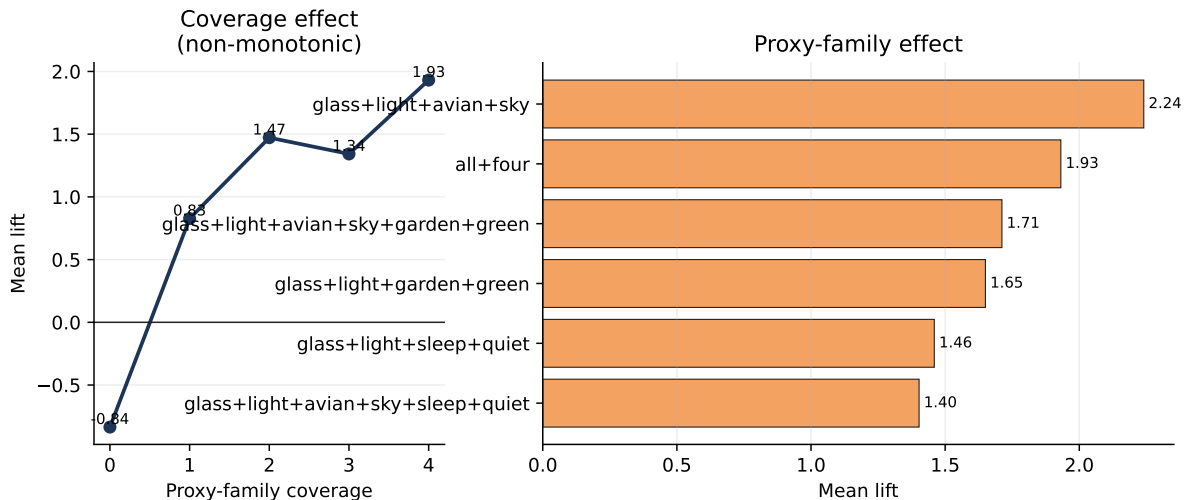


Figure 5: Proxy-family geometry. Coverage contributes to lift, but the strongest effect comes from proxy neighborhoods that best match the target basin.

The resulting interpretation is that an effective HDA is a compact hard-token attractor: a discrete prefix whose value comes from composed token evidence that moves the model into a useful high-dimensional basin. The comparison to compact latent or embedding-space representations is an analogy for the kind of movement being measured, not evidence that HDA strings inherit the guarantees or mechanisms of those methods [5, 6, 4].

7 Activation-Space Diagnostics

The same claim can be represented geometrically. Without an HDA, the model begins decoding from a broad conditional region. A useful HDA moves the pre-decode state toward a task-relevant basin; a tokenizer-matched random string is not expected to make the same directed move. Figure 6 is a conceptual visualization of this mechanism, not a substitute for the measured evaluations above.

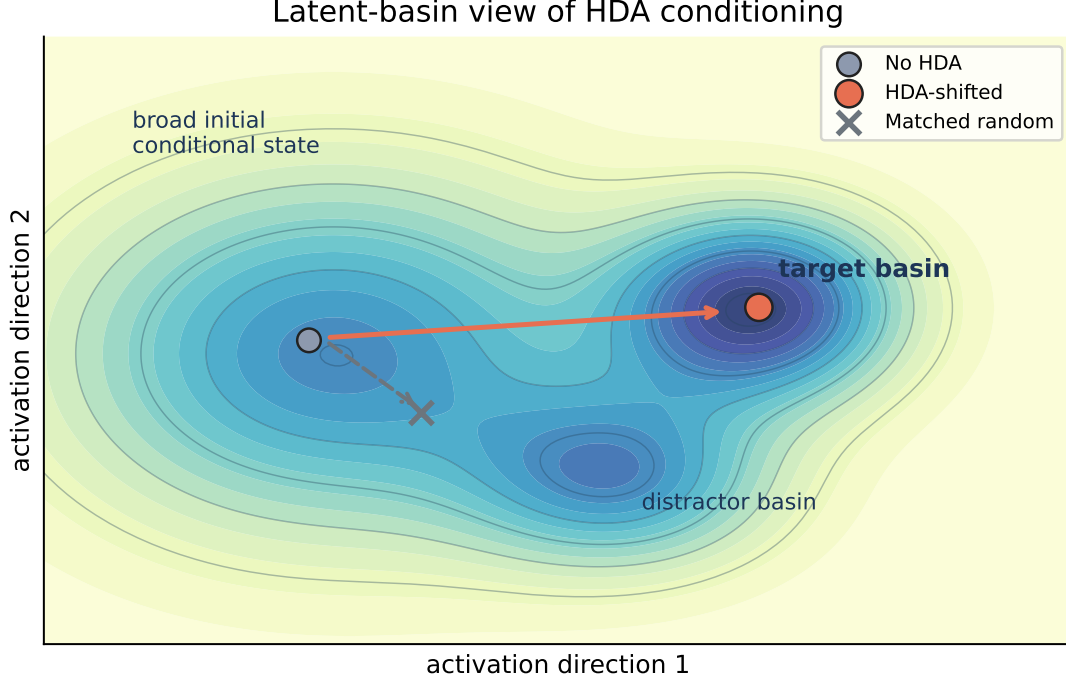


Figure 6: Conceptual activation-space view. The HDA is useful when it moves the pre-decode state toward a target basin before the final answer is sampled.

Activation-space diagnostics from Gemma latent-zone evaluations provide an additional internal measurement. In the symbolic-transform diagnostic, class-vector alignment separates in later layers: the good-HDA alignment exceeds the control alignment by $+0.0839$ at layer 40 and $+0.0886$ at layer 41. These diagnostics are supportive rather than dispositive: the proposed mechanism predicts that effective HDA prefixes should leave a measurable trace in hidden-state geometry before or during the controlled decode. The diagnostic role is similar in spirit to embedding-space translation and latent-action work, but the citation is used only to situate the representation-movement framing; the TSCE-specific claim rests on the matched-control measurements in this paper [5, 6].

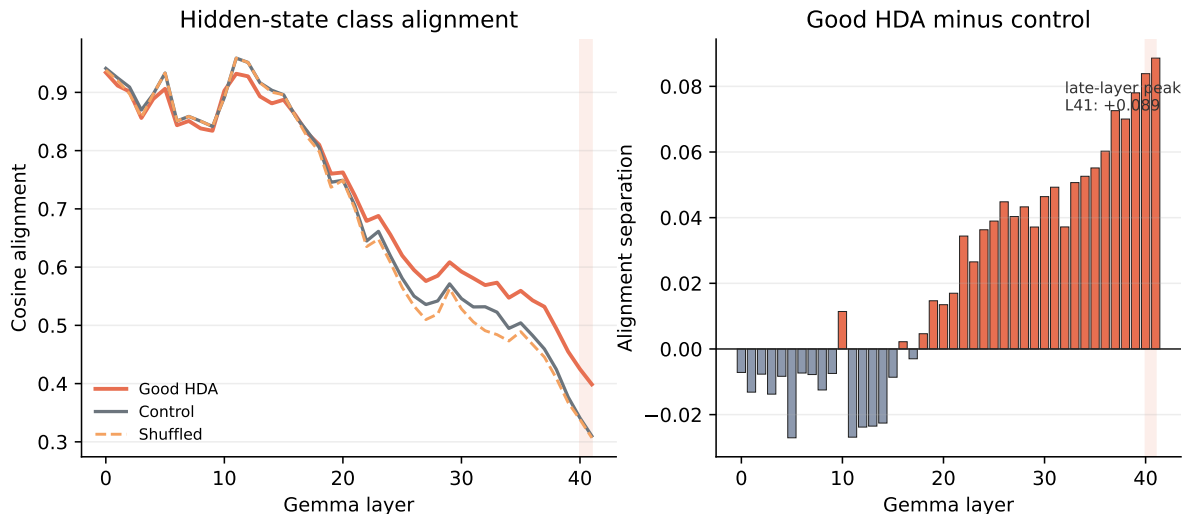


Figure 7: Activation-space diagnostic from a Gemma latent-zone evaluation. Good-HDA class alignment separates from controls in late layers, consistent with a token-mediated hidden-state shift.

8 Scope and Limitations

The claim is bounded in eight ways.

- TSCE does not show that hard-token HDA strings are as expressive as arbitrary continuous activation edits.
- TSCE does not require vector patching to work. Vector patching tests whether a detached mean activation delta can replace the token pathway; TSCE claims only token-mediated conditioning through the normal model interface.
- The baseline-vs-best outcome summary is an upper-envelope view across heterogeneous configurations, not a single fixed HDA generation recipe. The fixed compact Gemma `latent_bias` validation in Table 3 is neutral, so the present outcome evidence should not be read as proving a ready fixed production policy.
- The current HDA generation policy is reproducible, but not claimed to be optimal. Learned diffusion and reinforcement-learning policies remain separate engineering extensions.
- TSCE does not imply that every opaque string is useful. Random matched strings fail in the mechanistic test.
- Surface-form robustness does not imply scaffold invariance. The HDA must still be placed in a conditioning context that the model uses before answer decoding.
- TSCE does not imply that proxy coverage is monotonic. Coverage contributes to lift, but family geometry matters.
- TSCE does not guarantee uniform gains across all models and tasks. The expected benefit depends on whether the HDA produces useful pre-decode movement.

- TSCE is not benchmarked here against the latest agentic scaffolds, search-heavy agents, or broader test-time compute methods [12, 8, 14].

These limitations are part of the proposed mechanism. The claim does not require every HDA or every geometry diagnostic to succeed. It requires a link between outcome lift and a measurable pre-decode intervention, and the controlled token-causality evaluation supplies direct evidence for that link. Responsible deployment also requires standard provenance, access-control, and monitoring practices for generated control strings, consistent with broader system-card guidance for deployed language models [9].

9 Conclusion

TSCE is a two-step control procedure: the first step generates a Hyper-Dimensional Anchor, and the second step decodes the final answer under that anchor. The proposed mechanism is that reliability improves when the HDA shifts the model into a more task-appropriate pre-decode region.

The black-box evaluations measure the downstream effect. The controlled token-causality evaluation measures the proposed mediator: task-relevant HDA strings shift pre-generation probe margins while matched random strings do not. Together, these results support a discrete-control-prefix account of HDA conditioning.

The neutral fixed-policy Gemma run is part of the conclusion, not an exception to hide. Some anchor policies produce large effects; others are neutral. This makes the practical claim narrower and more useful. TSCE should be read as a framework for discovering and validating control prefixes, not as a one-size-fits-all recipe.

For industrial teams, the main implication is not that TSCE replaces evaluation, routing, retrieval, or agentic scaffolds. It is that reliability work can add a low-friction control layer before the final answer without fine-tuning, model hosting changes, or invasive activation access. A CTO or automation team can A/B test TSCE as a wrapper around existing prompts, measure token and latency overhead directly, and keep the intervention auditable because the HDA is an ordinary logged input string. The evidence here supports testing that wrapper where failures look like underconstrained decoding, format drift, or brittle instruction adherence; it does not support deploying arbitrary opaque strings without matched controls and task-level acceptance tests.

A Result Corpus Inventory

Table 7 is the category rollup generated by the result-inventory script. The script writes the per-file CSV inventory and summary JSON under `docs/`.

Artifact class	Files	Count	Interpretation
Black-box pass-rate artifacts	148	19,245	Includes summaries, per-task detail rows, generated answer rows, cost/latency rows, and diagnostics.
10k anchor-mining generation artifacts	6	33,125	Base prompts, expanded prompts, batch requests, batch outputs, and request index rows.
10k anchor-mining parsed/reranker artifacts	15	28,184	Parsed candidates, shortlists, training candidates, holdout predictions, and reranker scores.
Anchor-mining smoke/repair artifacts	41	3,008	Smaller mining, repair, and repair-run records.
HDA reinforcement-learning artifacts	25	8,757	Policy-training and evaluation traces.
Controlled token-causality artifacts	16	4,082	Probe rows plus generated/reranked controlled-anchor rows and summaries.
Gemma latent-zone artifacts	33	2,056	Latent-zone candidate, causal, family, shell, and activation diagnostics.
Gemma comparison/family/fixed-policy artifacts	131	1,230	Gemma comparison, family, fixed-policy, task-set, summary, and activation-capture artifacts.
API candidate-generation artifacts	8	380	Smaller API and format-control candidate-generation runs.
Run logs, verification, and other result artifacts	56	749	Run logs, verification logs, small evaluations, manifests, and miscellaneous result records.
Total result-corpus artifacts	479	100,816	Counted rows, records, or log lines across the result tree; heterogeneous artifact inventory, not a pooled independent-trial denominator.

Table 7: Result corpus inventory for `tsce_agent_demo/results`. The counts distinguish comparable accuracy trials from supporting generation, policy, and mechanistic artifacts.

B Reproducibility Notes

- 300-task GPT-3.5 black-box evaluation summary.

- Additional 100-task, 200-task, 1,000-task, temperature-sweep, fixed-policy Gemma, and model-variant black-box summaries.
- Fixed-policy Gemma result path:
`tsce_agent_demo/results/gemma_fixed_latent_bias_compact_heldout_n100_seed260426`.
- Format-robustness sweep path:
`tsce_agent_demo/results/hda_anchor_format_full_sweep_seed260501_u69_k16_combined`.
- Reference wrapper entry point: `tsce_agent_demo/tsce_chat.py`; diffusion anchor service entry point: `tsce_agent_demo/diffusion_api.py`.
- Controlled composite-target proxy-family token-causality analysis.
- Naturalistic story-theme token-causality analysis for *forgiveness after betrayal*.
- HDA perturbation and property-analysis outputs for matched-control comparisons.
- Figure-generation code used to render all paper figures.

C HDA Generation Policies

The current prompt-family policy samples candidate anchors from a named family and selects one candidate with a heuristic validator. The validator favors compact alphabetic strings with high uniqueness, low repetition, low prompt overlap, no digits, no punctuation, no instruction vocabulary, and low sentence-likeness. The fixed-policy Gemma validation uses the `latent_bias` family, compact 4–8 token anchors, two candidates, anchor temperature 1.25, top- $p = 0.95$, and the heuristic selector. Runs that sweep families or candidates record those settings in their result metadata.

This paper treats the HDA mechanism as the central claim and leaves learned HDA policies as a downstream engineering problem. Diffusion and reinforcement-learning policies are useful to the extent that they discover short HDA strings that reliably create the desired pre-decode movement. Such policies should be evaluated under the same standard used here: downstream task lift plus pre-generation movement against matched controls.

References

- [1] Nasim Borazjanizadeh and Steven T. Piantadosi. Reliable reasoning beyond natural language. arXiv preprint arXiv:2407.11373, 2024. <https://arxiv.org/abs/2407.11373>
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. Advances in Neural Information Processing Systems 33, 2020. <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [3] Valeriia Cherepanova and James Zou. Talking nonsense: Probing large language models’ understanding of adversarial gibberish inputs. arXiv preprint arXiv:2404.17120, 2024. <https://arxiv.org/abs/2404.17120>
- [4] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. arXiv preprint arXiv:2412.06769, 2024. <https://arxiv.org/abs/2412.06769>

- [5] Rishi Jha, Collin Zhang, Vitaly Shmatikov, and John X. Morris. Harnessing the universal geometry of embeddings. arXiv preprint arXiv:2505.12540, 2025. <https://arxiv.org/abs/2505.12540>
- [6] Chengxing Jia, Ziniu Li, Pengyuan Wang, Yi-Chen Li, Zhenyu Hou, Yuxiao Dong, and Yang Yu. Controlling large language model with latent action. Proceedings of the 42nd International Conference on Machine Learning, PMLR 267:27331–27372, 2025. <https://proceedings.mlr.press/v267/jia25e.html>
- [7] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 2024. <https://aclanthology.org/2024.acl-long.511/>
- [8] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. Advances in Neural Information Processing Systems 36, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html
- [9] OpenAI. GPT-4 system card. <https://cdn.openai.com/papers/gpt-4-system-card.pdf>, 2023.
- [10] Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, and Richard Socher. CTRL: A conditional transformer language model for controllable generation. arXiv preprint arXiv:1909.05858, 2019. <https://arxiv.org/abs/1909.05858>
- [11] Thomas M. Cover and Joy A. Thomas. Elements of information theory. Wiley, second edition, 2006. <https://doi.org/10.1002/047174882X>
- [12] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. International Conference on Learning Representations, 2023. <https://openreview.net/forum?id=1PL1NIMMrw>
- [13] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems 35, 2022. <https://arxiv.org/abs/2201.11903>
- [14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations, 2023. https://openreview.net/forum?id=WE_vluYUL-X
- [15] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. International Conference on Learning Representations, 2024. <https://openreview.net/forum?id=ph04CRkPdC>
- [16] Jeffrey Cheng and Benjamin Van Durme. Compressed chain of thought: Efficient reasoning through dense representations. arXiv preprint arXiv:2412.13171, 2024. <https://arxiv.org/abs/2412.13171>